Laboratorium 2 — Kryptografia historyczna

Środa, 14.10.2026, 12:15–14:00 · GE-b 510 · GR6, GR7
Pliki do zadań: cezar.bin · tekst_pl.txt · rc4.bin (udostępnione przez prowadzącego na zajęciach)

Zasady

Wykonaj trzy zadania w Pythonie 3. Łącznie możesz zdobyć **3 pkt**. Implementacje szyfrów i ataku napisz samodzielnie na podstawie materiałów; gotowe biblioteki kryptograficzne nie zastępują własnego kodu. Zadania są niezależne — gdy utkniesz, przejdź do następnego i oddaj również ukończone części.

Przygotowanie i samodzielność. Możesz przygotować w domu własny kod i notatki oraz korzystać z nich na zajęciach. Podczas zajęć nie wolno korzystać z modeli językowych (LLM), w tym czatów AI, asystentów programistycznych i podpowiedzi kodu opartych na LLM. Wyłącz te funkcje w edytorze przed rozpoczęciem pracy. Na zajęciach samodzielnie dostosowujesz i uruchamiasz kod, analizujesz wyniki oraz przygotowujesz raport. Musisz umieć wyjaśnić cały oddawany kod, także przygotowany wcześniej.

Wszystkie pliki z danymi czytaj i zapisuj **binarnie** (tryby rb i wb). Bajty, które mogą zawierać znaki sterujące, wypisuj przez repr(), np. print(repr(dane[:50])).

Zadanie 1. Cezar: odzyskaj przesunięcie — 1 pkt, około 15 minut

Użyj pliku **cezar.bin**. Zawiera tekst zaszyfrowany przez stałe przesunięcie w alfabecie 0–255: każdy bajt p zastąpiono bajtem (p + k) % 256. Dla kandydata k odzyskaj bajty według wzoru p = (c - k) % 256.

1. Sprawdź wszystkie 256 przesunięć. Dla każdego wyznacz udział bajtów odpowiadających drukowalnym znakom ASCII (32–126) lub tabulatorowi i końcom wiersza (9, 10, 13). Wypisz pięciu najlepszych kandydatów wraz z oceną i początkiem odszyfrowania; remisy uporządkuj rosnąco według k.
2. Spośród kandydatów wybierz przesunięcie dające sensowny angielski tekst i zapisz całe odszyfrowanie jako **cezar.txt**. W tym zadaniu wybór końcowy może być ręczny.
3. W raporcie podaj przesunięcie i krótko uzasadnij wybór. Wyjaśnij, dlaczego sama entropia histogramu wartości 0–255 nie rozstrzygnie, które przesunięcie jest poprawne.

Zadanie 2. Vigenère: sprawdź wpływ powtarzania klucza — 1 pkt, około 25 minut

Zaimplementuj szyfrowanie i odszyfrowanie **bajtowego wariantu Vigenère'a**. Dla niepustego klucza K o długości m stosuj c[i] = (p[i] + K[i % m]) % 256; odszyfrowanie ma odejmować wartość klucza modulo 256.

1. Zaszyfruj plik **tekst_pl.txt**, odczytany binarnie (zawiera polskie znaki w UTF-8), kluczem ASCII **ZAMEK**. Zapisz **vigenere.bin**. Odszyfruj wynik i sprawdź zgodność ze źródłem bajt po bajcie. Sprawdź też puste dane oraz dane zawierające bajty 0 i 255.
2. Zaszyfruj ciąg ASCII **BRAMABRAMABRAMA** kolejno kluczami **SZYFR** i **KOD**. Pokaż oba kryptogramy szesnastkowo (np. bytes.hex()), dzieląc każdy na trzy fragmenty po pięć bajtów.
3. W 2–3 zdaniach wyjaśnij, dlaczego w jednym przypadku powtarzają się całe pięciobajtowe fragmenty kryptogramu, a w drugim nie. Odnieś to do położenia powtarzanego tekstu względem klucza i informacji widocznej dla atakującego.

Zadanie 3. RC4: automatycznie odzyskaj krótki klucz — 1 pkt, około 50 minut

Użyj pliku **rc4.bin**. Zawiera angielski tekst zaszyfrowany RC4 kluczem złożonym z **dokładnie trzech małych liter ASCII a–z**. Szyfrowanie rozpoczęto od początku strumienia, bez odrzucania początkowych bajtów.

1. Na podstawie pseudokodu ISOD zaimplementuj RC4: KSA, PRGA i XOR z danymi. Każde wywołanie zaczyna od nowego stanu. Sprawdź, czy dwukrotne zastosowanie funkcji z tym samym kluczem odtwarza wejście.
2. Sprawdź wszystkie klucze. Dla każdego odszyfruj **pierwsze 512 bajtów**, oblicz entropię i utwórz ranking rosnący. Wypisz pięciu najlepszych kandydatów; przy remisie decyduje kolejność alfabetyczna. Automatycznie wybierz pierwszego — nie wpisuj klucza na stałe i nie kończ przeszukiwania po przekroczeniu umownego progu.
3. Wybranym kluczem odszyfruj cały plik **od początku i z nowym stanem RC4**. Zapisz **rc4.txt** bez zmieniania odzyskanych bajtów. Sprawdź sensowność tekstu oraz zgodność ponownego szyfrowania z kryptogramem.
4. W raporcie zapisz klucz, ranking pięciu kandydatów, liczbę prób, czas przeszukiwania oraz entropię całego kryptogramu i całego odszyfrowania. W 2–3 zdaniach wyjaśnij, dlaczego ponowne szyfrowanie może odtworzyć kryptogram również dla błędnego klucza i kiedy ranking entropii mógłby zawieść.

Oddanie — jeden pakiet do 14:00

Oddaj w miejscu wskazanym przez prowadzącego **nazwisko_nrindeksu_lab02.zip**:

- **cezar.py, vigenere.py, rc4.py** — programy uruchamiane odpowiednio jako: python cezar.py cezar.bin; python vigenere.py tekst_pl.txt; python rc4.py rc4.bin. Program Cezara może poprosić o wybór kandydata po wyświetleniu rankingu.
- **cezar.txt, vigenere.bin, rc4.txt** — wyniki zadań.
- **wyniki.md** — krótki raport, maksymalnie dwie strony: wyniki i wyjaśnienia wymagane w każdym zadaniu oraz rezultaty testów. Nie potrzeba opisów teorii ani zrzutów całego kodu.

Przy oddaniu przygotuj się na krótkie pytanie o swój kod. Sam tekst jawny lub sam zrzut ekranu nie zastępuje programu. Nie dołączaj środowiska wirtualnego ani kopii materiałów kursu.